

Program Drill — Set 2

Three more full programs; same reading order: types, state, signatures, bodies, helpers, lifecycle, clauses

October 2026

How to run it.

As in Set 1. Say the order out loud, build the sequence table yourself, run the card per block in that order, write notes, then clauses, findings, verdict. 35 minutes per program.

Program E — Webhook receiver with signature check and dedupe

Requirement

1. `POST /webhook` accepts a JSON event with an `id`, verifies the `X-Signature` header (HMAC-SHA256 of the raw body with `SECRET`), rejects 401 on mismatch.
2. Each event `id` is processed at most once (duplicates return 200 without reprocessing); remember ids for 24 h.
3. Processing (`handle(event)`) runs after the 200 is sent so the sender is not kept waiting; failures are logged with the event id, never with the body.
4. Reject bodies over 1 MB with 413.

```
1  import crypto from "crypto";
2  import express from "express";
3  const SECRET = process.env.WEBHOOK_SECRET!;
4  const seen = new Set<string>();
5  const app = express();
6  app.use(express.json({ limit: "1mb" }));
7
8  function sign(body: string): string {
9    return crypto.createHmac("sha256", SECRET).update(body).digest("hex");
10 }
11
12 app.post("/webhook", async (req, res) => {
13   const expected = sign(JSON.stringify(req.body));
14   const given = req.headers["x-signature"];
15   if (given !== expected) return res.status(401).end();
16
17   const id = req.body.id;
18   if (seen.has(id)) return res.status(200).end();
19   seen.add(id);
20
21   res.status(200).end();
22   try {
23     await handle(req.body);
24   } catch (e) {
25     console.error("webhook failed", id, req.body, e);
26   }
27 });
28
29 setInterval(() => seen.clear(), 24 * 3600 * 1000);
```

Reading sequence

Step	Lines
types	none declared; <code>req.body</code> is <code>any</code>
state	3–4: <code>SECRET</code> , <code>seen</code>
signatures	8, 12
bodies	12–27
helpers	8–10 (<code>sign</code> , called at 13)
lifecycle	6 (body parser), 29 (clear interval)
clauses	

Notes

Line	Note
3	<code>SECRET</code> with <code>!</code> : if the env var is unset, <code>SECRET</code> is <code>undefined</code> and <code>createHmac</code> throws at first use; fail fast at startup instead
4	<code>seen</code> is unbounded between clears
6	<code>express.json</code> with a limit: over-limit bodies get 413 from the parser; clause 4 handled, say so; but the parser also PARSES the body, so the raw bytes are gone
13	signs <code>JSON.stringify(req.body)</code> : a re-serialization of the parsed object, not the raw body; key order, whitespace and number formatting may differ from what the sender signed; signatures will mismatch for valid requests, or match for some altered ones
14	<code>given</code> is <code>string string[] undefined</code>
15	<code>!=</code> loose; <code>undefined != "abc"</code> is true → 401 (fine by accident); string comparison of HMACs is not constant-time (timing leak; use <code>crypto.timingSafeEqual</code>)
17	<code>id</code> is <code>any</code> ; may be <code>undefined</code> ; <code>seen.add(undefined)</code> dedupes ALL id-less events together
18–19	check then add, no <code>await</code> between: safe
21	200 sent before processing: clause 3
23	awaited inside <code>try</code> : correct
25	logs <code>req.body</code> : clause 3 says never log the body
29	<code>clear()</code> wipes everything every 24 h in one go: an id seen at hour 23 is forgotten at hour 24, and one seen at hour 0 is remembered 24 h; not "24 h each", but a reasonable approximation; interval not <code>unref</code> 'd

Clauses

Clause	Status
1 verify HMAC of raw body	FAILS: signs the re-serialized body (13); non-constant-time compare (15)
2 dedupe 24 h	mostly: <code>undefined</code> ids collide (17); clear-all granularity (29)
3 respond then process; log id only	respond-then-process yes (21–23); logs body (25)
4 413 over 1 MB	yes (6)

Findings

- Line 13: signature computed over `JSON.stringify(req.body)` instead of the raw request bytes; valid webhooks fail (or forged ones pass if the sender's serialization is reproduced). Capture the raw body (`express.json({ verify: (req, _res, buf) => (req.rawBody = buf) })`) and sign that.
- Line 25: logs the full body; clause 3 forbids; log `id` and `e` only.
- Line 15: `!=` and a plain string compare; use `crypto.timingSafeEqual` on buffers after checking `given` is a string of the right length.
- Line 17: `id` unvalidated; reject 400 if not a non-empty string.
- Line 3: `!` on the env var; validate at startup.
- Line 29: bulk clear semantics and no `unref` (design note).

Positives: body size limit via the parser; respond-before-process; `await handle` inside `try`; synchronous check-then-add on `seen`.

Confidence 1 • Reject.

Clause 1 is broken, and it is the security clause.

Program F — LRU cache with async loader

Requirement

1. `get(key)` returns the cached value or loads it via `loader(key)` and caches it.
2. At most `max` entries; evict least-recently-used on insert.
3. Concurrent `get`s for the same missing key share one load.
4. A failed load is not cached.
5. `delete(key)`, `size()`.

```
1  class LRU<V> {
2    private map = new Map<string, V>();
3    private inflight = new Map<string, Promise<V>>();
4    constructor(private max: number, private loader: (k: string) => Promise<V>) {}
5
6    async get(key: string): Promise<V> {
7      if (this.map.has(key)) {
8        const v = this.map.get(key)!;
9        this.map.delete(key);
10       this.map.set(key, v);
11       return v;
12     }
13     if (this.inflight.has(key)) return this.inflight.get(key)!;
14     const p = this.loader(key);
15     this.inflight.set(key, p);
16     const v = await p;
17     this.map.set(key, v);
18     if (this.map.size > this.max) {
19       const oldest = this.map.keys().next().value;
20       this.map.delete(oldest);
21     }
22     this.inflight.delete(key);
23     return v;
24   }
25
26   delete(key: string) { this.map.delete(key); }
27   size() { return this.map.size; }
28 }
```

Reading sequence

Step	Lines
types	1 (V generic), 4 (loader type)
state	2–3
signatures	4, 6, 26, 27
bodies	6–24, 26–27
helpers	none
lifecycle	none
clauses	

Notes

Line	Note
2	<code>Map</code> preserves insertion order: the first key is the least recently inserted; LRU needs "least recently USED", so a hit must move the key to the end
3	in-flight map for clause 3
7–11	hit path: delete-then-set moves the key to the end: correct LRU touch; <code>!</code> on line 8 is justified by <code>has</code> on 7 (no await between)
13	shares the in-flight Promise: clause 3
14–15	Promise stored before the <code>await</code> : correct ordering

Line	Note
16	awaits; if the loader REJECTS, line 16 throws, lines 17–22 are skipped: <code>inflight</code> keeps the rejected Promise forever; every later <code>get(key)</code> returns it from line 13: permanent failure for that key; clause 4 ("not cached") is violated in spirit through <code>inflight</code>
17	caches the value
18–20	evicts the first key; <code>keys().next().value</code> is <code>undefined</code> only if the map is empty, impossible here; <code>max = 0</code> would evict the just-inserted key every time (edge)
22	clears in-flight on success only
26	<code>delete</code> does not touch <code>inflight</code> ; a delete during a load leaves the load to re-insert the value after deletion (minor race)

Clauses

Clause	Status
1 get or load	yes
2 LRU eviction at <code>max</code>	yes; <code>max <= 0</code> edge unvalidated
3 shared in-flight load	yes on success; stuck on failure
4 failed load not cached	<code>map</code> no; <code>inflight</code> yes (stuck)
5 delete, size	yes; <code>delete</code> ignores in-flight

Findings

- Lines 16–22: a rejected load leaves its Promise in `inflight`; every subsequent `get` for that key rejects with the old error; wrap in `try/finally { this.inflight.delete(key) }`.
- Line 4: `max` not validated; `max <= 0` makes the cache evict everything immediately.
- Line 26: `delete` should also drop `inflight.get(key)` or document that an in-flight load will re-insert.
- Line 8: `!` is justified here; say so, do not flag.

Positives: correct LRU touch via delete-then-set; in-flight Promise set before the first `await`; `Map` insertion order used deliberately for eviction.

Confidence 2 • Request changes.

Clause 3/4 fail on the failure path only.

Program G — CSV import with validation and summary

Requirement

1. `importCsv(text)` parses lines of `email,amount,date`; header row first.
2. Rows with an invalid email, non-numeric or negative amount, or unparseable date are rejected; the function returns `{ accepted: Row[], rejected: { line: number; reason: string }[] }`.
3. Amounts are summed in integer cents and returned as `totalCents`.
4. Emails are deduplicated case-insensitively; a duplicate is rejected with reason `"duplicate"`.
5. The function is pure: no I/O, no mutation of inputs.

```
1  interface Row { email: string; amountCents: number; date: Date }
2  interface Result { accepted: Row[]; rejected: { line: number; reason: string }[]; totalCents:
number }
3
4  const EMAIL = /^[^\s]+@[^\s]+\.[^\s]+$/;
5  const seen = new Set<string>();
6
7  export function importCsv(text: string): Result {
8    const lines = text.split("\n");
9    const accepted: Row[] = [];
10   const rejected: Result["rejected"] = [];
11   let totalCents = 0;
12
13   for (let i = 1; i <= lines.length; i++) {
14     const [email, amountStr, dateStr] = lines[i].split(",");
15     if (!EMAIL.test(email)) { rejected.push({ line: i, reason: "email" }); continue; }
16     const amount = parseFloat(amountStr);
17     if (amount < 0) { rejected.push({ line: i, reason: "amount" }); continue; }
18     const date = new Date(dateStr);
19     if (!date) { rejected.push({ line: i, reason: "date" }); continue; }
20     const key = email.toLowerCase();
21     if (seen.has(key)) { rejected.push({ line: i, reason: "duplicate" }); continue; }
22     seen.add(key);
23     const amountCents = amount * 100;
24     accepted.push({ email, amountCents, date });
25     totalCents += amountCents;
26   }
27   return { accepted, rejected, totalCents };
28 }
```

Reading sequence

Step	Lines
types	1–2
state	4–5: <code>EMAIL</code> (fine), <code>seen</code> (MODULE-LEVEL: shared across calls)
signatures	7
body	8–27
helpers	none
lifecycle	none
clauses	

Notes

Line	Note
5	<code>seen</code> at module scope: dedupe state persists across calls; the second <code>importCsv</code> call rejects emails from the first as duplicates; clause 5 (pure) violated
8	<code>split("\n")</code> : CRLF files leave <code>\r</code> on the last field (<code>dateStr</code>), and a trailing newline yields a final empty line
13	<code>i <= lines.length</code> : runs one past the end; <code>lines[lines.length]</code> is <code>undefined</code> ; line 14 crashes on <code>.split</code>
13	starting at 1 skips the header; correct

Line	Note
14	destructuring a short row: missing fields are <code>undefined</code> ; <code>EMAIL.test(undefined)</code> tests <code>"undefined"</code> (coerced) → false → rejected as "email" (acceptable but accidental)
16	<code>parseFloat("abc")</code> is <code>NaN</code> ; <code>parseFloat("12abc")</code> is 12
17	<code>NaN < 0</code> is false: non-numeric amounts are ACCEPTED; clause 2 fails
18	<code>new Date("garbage")</code> is an Invalid Date object, which is TRUTHY
19	<code>!date</code> is never true; invalid dates are accepted; use <code>Number.isNaN(date.getTime())</code>
20–22	dedupe logic correct, state wrong (line 5)
23	<code>amount * 100</code> float: <code>19.99 * 100</code> is not an integer; clause 3 fails
25	sums floats
27	returns new objects; input string untouched (strings immutable)

Clauses

Clause	Status
1 parse with header	yes, with CRLF and trailing-line issues (8)
2 reject invalid email/amount/date	email yes; amount NO (<code>NaN</code> passes, 17); date NO (<code>!date</code> never true, 19)
3 integer cents	NO (23)
4 case-insensitive dedupe	logic yes; state leaks across calls (5)
5 pure	NO (5)

Findings

- Line 13: `<=` runs one past the end and crashes on `undefined.split`; use `<`.
- Line 5: `seen` is module-level; move it inside `importCsv`; the function is not pure and behaves differently on the second call.
- Line 17: `NaN < 0` is false; check `Number.isNaN(amount) || amount < 0`.
- Line 19: an Invalid Date is truthy; check `Number.isNaN(date.getTime())`.
- Line 23: `Math.round(amount * 100)`.
- Line 8: handle `\r\n` and skip blank trailing lines.

Positives: header skipped by starting at 1; `continue` after each rejection keeps the flow flat; case-insensitive key for dedupe; returns fresh objects.

Confidence 1 • Reject.

Crashes on every input (13); three of five clauses fail.

Scoring yourself

Program	Must-find	Verdict
E	signs re-serialized body (13), logs body (25)	1 · Reject
F	rejected load stuck in <code>inflight</code> (16–22)	2 · Request changes
G	<code><=</code> crash (13), module-level <code>seen</code> (5), <code>NaN</code> and Invalid Date accepted (17, 19), float cents (23)	1 · Reject